Machine Coding: Virtualized List

Problem: Build a virtualized (windowed) list that can show 100,000+ rows smoothly by rendering only the rows on screen.
Tech stack: React 19 + TypeScript (Vite)
Time limit: 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements:** what must it do, what must it handle, what is out of scope. Ask before you assume.
2. **List the interactions:** every user action and what should happen. This becomes your checklist.
3. **Sketch the architecture:** a few components with one job each. Start simple.
4. **Design the state:** the smallest set of values that describes the UI. Everything else is derived.
5. **Define data and contracts:** item shape, props, the public API of the component.
6. **Trace one flow end to end:** event, state change, render. If you can't trace it, the design is not ready.
7. **Find the hard parts:** solve them on paper before you type.
8. **Build in layers:** a working thing first, then add one feature at a time, keeping it working.
9. **Cover states and edge cases:** empty, error, boundaries, rapid input.
10. **Wrap up:** say what you handled, the trade-offs, and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, clarifying questions, assumptions	5 min
Use cases, architecture, state design	7 min
Core windowing math + plain scrolling list	15 min
Overscan, keys, perf (re-render only on row change)	8 min
`scrollToIndex`, filter, empty state, a11y	15 min
Edge cases, cleanup, walk the interviewer through it	10 min

Think out loud

Say every decision and why as you make it: "I'll store the first row index instead of scrollTop so React skips re-renders inside a row." The interviewer grades your reasoning, not only the final code. If you get stuck, say what you are trying and what you expect to see.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show a list of a large number of items (100,000) inside a fixed-height scroll container.
- Only the rows inside the viewport (plus a small buffer) exist in the DOM at any time.
- The scrollbar must look and behave like the full list is there (correct size and position).
- Scrolling (mouse, trackpad, keyboard, dragging the scrollbar) shows the right rows with no gaps.
- **Jump to row:** type a row number and scroll straight to it.
- **Filter:** type text to narrow the list; the list updates and starts at the top.
- Show an empty state when nothing matches.

Non-functional requirements

Area	What it means here
Performance	Constant DOM size (~20 rows) no matter how many items. No re-render on every pixel of scroll. 60 fps scrolling.
Scalability	100k items work out of the box; 1M is possible with a note on browser max element height.
Error handling	Invalid jump input shows a message. Out-of-range indices are clamped, never crash.
Accessibility	Container is focusable (keyboard scrolling), `role="list"`, each row has `aria-setsize` / `aria-posinset` so screen readers know "row 772 of 100,000".
Reusability	`VirtualList<T>` is generic: caller passes `items`, `renderItem`, `getKey`. It knows nothing about the data.
Extensibility	Math lives in a pure function and a hook, so variable heights, horizontal lists or grids can be added later.

Clarifying questions to ask

1. Do all rows have the **same height**, or can they vary (e.g. wrapped text)?
2. How many items: thousands, hundreds of thousands, millions? Is the data all in memory or paged from an API?
3. Is the container a **fixed height** or should it fill the window / parent?
4. Do we need extra features: jump to index, filter/search, infinite loading, selection?
5. Can I use a library like `react-window`, or should I build the windowing myself?
6. Which browsers? (Affects the max scroll height limit and `ResizeObserver` support.)

Assumptions we build with

- **Fixed row height** (56px). Variable heights are covered as a challenge and a follow-up.
- 100,000 items generated in memory, each with a stable `id`.
- Fixed container height (480px), passed as a prop.

- Build the windowing ourselves, no library.
- Features: smooth scroll, jump to row, filter, empty state.
- Modern browsers.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	100,000 items exist, ~15 rows rendered (visible + overscan), scrollbar sized for 100,000 × 56px.
Scroll down slowly	Rows below appear, rows above leave the DOM. DOM row count stays ~20.
Scroll within the same row (a few pixels)	No React re-render (first visible index did not change).
Fast scroll / fling	Overscan rows cover the gap; no blank flash.
Drag scrollbar to the bottom	Last row ("Item 100000") is fully visible; no blank rows past the end.
Keyboard: focus list + Arrow / PageDown / End	List scrolls (native behavior because the container is focusable).
Jump to row 777	List scrolls so row 777 is at the top; rows render immediately.
Jump to 0, -5, 3.5, empty, or > count	Error: "Enter a row between 1 and N". No scroll.
Type a filter "9999"	Count shows "19 items", list resets to top, only matching rows.
Filter with no matches	Empty state: `No items match "zzz"`.
Clear the filter	Full list back, starting at the top.
Screen reader on a row	Announces it as list item 772 of 100,000.

This table is the implementation checklist and the manual test plan.

---

3. Design the architecture

Module	Responsibility
`getVisibleRange` (pure util)	Given the first visible index, viewport height, row height, count and overscan, return `{ start, end }`. No React.
`useVirtualList` (hook)	Owns scroll state. Exposes `containerRef`, `onScroll`, `range`, `totalHeight`, `scrollToIndex`.
`VirtualList<T>` (component)	Renders the scroll container, the full-height spacer, and only the rows in range. Exposes `scrollToIndex` via `ref`.
`makeItems` (data)	Generates mock items with stable ids.
`App`	Page: filter input, jump form, count, and the `VirtualList` with a row renderer.

```
App
├── filter input        (query state)
├── jump form           (jumpTo, jumpError state)
├── item count          (derived: items.length)
└── VirtualList<ListItem>  ref -> { scrollToIndex }
    └── useVirtualList   (firstIndex state)
        └── getVisibleRange  (pure math)
    DOM:
    div.vlist  (fixed height, overflow: auto, onScroll)
    └── div.vlist__spacer  (height = count × rowHeight)
        ├── div.vlist__row  translateY(start × h)
        ├── ...             (~20 rows only)
        └── div.vlist__row  translateY((end-1) × h)
```

Start with everything in one component; pull out the math as a pure function as soon as it works (it is the part you want to test), then the hook once the component gets busy.

---

4. Design the state / data model

State or field	Owner	Purpose
`firstIndex`	`useVirtualList`	Index of the row at the top edge of the viewport. The only scroll state.
`query`	`App`	Filter text.
`jumpTo`	`App`	Controlled value of the jump input.
`jumpError`	`App`	Validation message for the jump input.
`items`	`App` (derived, `useMemo`)	`ALL_ITEMS` filtered by `query`.
`range`	derived in the hook	`{ start, end }` from `getVisibleRange`.
`totalHeight`	derived in the hook	`items.length × itemHeight`, the spacer height.

```
// useVirtualList.ts
const [firstIndex, setFirstIndex] = useState(0);
const range = getVisibleRange({ firstIndex, viewportHeight: height, itemHeight, itemCount, overscan
});
const totalHeight = itemCount * itemHeight;

// App.tsx
const [query, setQuery] = useState('');
const [jumpTo, setJumpTo] = useState('');
const [jumpError, setJumpError] = useState('');
const items = useMemo(() => filter(ALL_ITEMS, query), [query]);
```

Senior note:

- **Derive, don't store.** `range`, `totalHeight`, the visible slice and `items` are all computed during render. Storing them would let them go out of sync with their inputs.
- **Store `firstIndex`, not `scrollTop`.** `scrollTop` changes every pixel; `firstIndex` changes once per row. React bails out of `setState` when the value is the same, so scrolling inside a row costs zero

renders.

- **Ownership:** scroll state belongs to the list (the parent never needs it). Filter and jump input belong to the page. The parent controls scrolling only through the imperative `scrollToIndex` handle, the same pattern `react-window` uses.

---

5. Design the data structures and contracts

Example data

```
type ListItem = { id: string; title: string; subtitle: string };

// makeItems(100_000)
[
  { id: 'item-0', title: 'Item 1', subtitle: 'Design task' },
  { id: 'item-1', title: 'Item 2', subtitle: 'Frontend task' },
  // ...
  { id: 'item-99999', title: 'Item 100000', subtitle: 'DevOps task' },
]
```

Why stable ids: rows are created and destroyed all the time while scrolling, and the filter changes which item sits at which index. Keying by `item.id` (not the array index) means React keeps each row's DOM and state with its item, not with its slot.

Component contract

```
type VirtualListProps<T> = {
  items: T[];                                  // full list, any length
  itemHeight: number;                          // fixed px height of each row
  height: number;                              // px height of the viewport
  overscan?: number;                           // extra rows above/below (default 5)
  getKey: (item: T) => React.Key;              // stable key per item
  renderItem: (item: T, index: number) => React.ReactNode;
  emptyMessage?: React.ReactNode;              // shown when items is empty
  ariaLabel: string;                           // accessible name of the list
  ref?: React.Ref<VirtualListHandle>;          // React 19: ref is a normal prop
};

type VirtualListHandle = { scrollToIndex: (index: number) => void };
```

Pure function contract

```
getVisibleRange({ firstIndex: 100, viewportHeight: 500, itemHeight: 50, itemCount: 1000, overscan: 5
})
// -> { start: 95, end: 116 }   (end is exclusive; 10 visible + 1 partial + 5 + 5 overscan)
```

There is no real API. If the data came from a server, the contract would be a page endpoint like `GET /items?offset=2000&limit=100 -> { items, total }`; `total` sizes the spacer before the data arrives.

---

6. Trace the data flow

Interaction: the user scrolls down from row 0 to row 3.

```
User scrolls the container (wheel / drag / keyboard)
        ↓
Browser updates div.vlist scrollTop = 190 (it scrolls the tall spacer natively)
        ↓
onScroll(e) → setFirstIndex(Math.floor(190 / 56)) = setFirstIndex(3)
        ↓  (scrollTop 170..223 all give 3 → React bails out, no render)
React re-renders VirtualList once
        ↓
getVisibleRange({ firstIndex: 3, viewportHeight: 480, itemHeight: 56, itemCount: 100000, overscan: 5
})
   visibleCount = ceil(480/56) + 1 = 10
   start = max(0, 3 - 5) = 0,  end = min(100000, 3 + 10 + 5) = 18
        ↓
items.slice(0, 18) → map to rows, each with transform: translateY(index × 56px)
        ↓
React diffs by key: rows 0–14 stay, rows 15–17 are added
        ↓
Screen shows rows 3–11 in the viewport; the scrollbar thumb sits at 190 / 5,600,000
```

Interaction: jump to row 777.

```
Submit form → validate 1 ≤ 777 ≤ items.length → listRef.current.scrollToIndex(776)
        ↓
container.scrollTo({ top: 776 × 56 })  +  setFirstIndex(776)
        ↓
Render rows 771–791 immediately (no wait for the async scroll event)
        ↓
Scroll event arrives → setFirstIndex(776) → same value → no extra render
```

---

7. Important technical challenges

Challenge 1: Rendering only what is visible while keeping a real scrollbar

Problem: If you render only 20 rows, the container's content is 20 × 56 = 1,120px tall. The scrollbar thinks the list is tiny and the user can't scroll to row 50,000.

Solution: Two layers. A **spacer** div with the full height (`count × rowHeight`) gives the browser the real scroll range. Inside it, the few visible rows are **absolutely positioned** at `index × rowHeight` . The browser does the scrolling natively; we only decide which rows to mount.

```
<div className="vlist" style={{ height }} onScroll={onScroll}>
  <div className="vlist__spacer" style={{ height: totalHeight }}>
    {visible.map((item, i) => {
      const index = range.start + i;
      return (
        <div key={getKey(item)} className="vlist__row"
             style={{ height: itemHeight, transform: `translateY(${index * itemHeight}px)` }}>
```

```
          {renderItem(item, index)}
        </div>
      );
    })}
  </div>
</div>
```

`transform` instead of `top`: moves the row on the compositor and does not trigger layout.

Challenge 2: The windowing math and its off-by-one traps

Problem: Scenario: viewport 500px, rows 50px, `scrollTop = 20`. Rows 0–9 cover 20..500, but row 10 also peeks in from 500 to 520. `ceil(500/50) = 10` rows misses it → a white strip at the bottom. Second trap: the user filters 100,000 rows down to 19 while scrolled to row 50,000. The stale index gives `start = end = 19` → zero rows rendered until the scroll event catches up.

Solution: `+1` row for the partial row; clamp the first index to the last valid page; clamp both ends to `[0, itemCount]`. Keep it a pure function so you can assert every edge.

```
export function getVisibleRange({ firstIndex, viewportHeight, itemHeight, itemCount, overscan }:
RangeInput): Range {
  if (itemCount <= 0 || itemHeight <= 0) return { start: 0, end: 0 };
  const maxFirst = Math.max(0, Math.floor(itemCount - viewportHeight / itemHeight));
  const first = Math.min(firstIndex, maxFirst);
  const visibleCount = Math.ceil(viewportHeight / itemHeight) + 1;
  const end = Math.min(itemCount, first + visibleCount + overscan);
  const start = Math.max(0, first - overscan);
  return { start, end };
}

// assert-based self-check
assert.deepEqual(r(0),           { start: 0,   end: 11 });
assert.deepEqual(r(100, 1000, 5),{ start: 95,  end: 116 });
assert.deepEqual(r(5000, 19, 5), { start: 4,   end: 19 });  // stale index, not blank
assert.deepEqual(r(0, 0),        { start: 0,   end: 0 });
```

Challenge 3: Scroll performance (renders per scroll and blank flashes)

Problem: A scroll event fires about once per frame. If you `setScrollTop(e.currentTarget.scrollTop)`, React re-renders on every pixel, even when the visible rows are the same. On a fast fling, the browser paints the new scroll position before React renders the new rows → a blank flash.

Solution:

- Store the **row index**. `setFirstIndex(same value)` is a no-op for React, so renders drop from "every pixel" to "every row".
- **Overscan** 5 rows above and below. The browser can scroll ~5 rows before React must catch up.
- **Stable keys** so rows that stay on screen are not re-created.
- Keep `renderItem` cheap; wrap heavy rows in `memo` if profiling shows it.

```
const [firstIndex, setFirstIndex] = useState(0);
const onScroll = (e: UIEvent<HTMLDivElement>) => {
```

```
  setFirstIndex(Math.floor(e.currentTarget.scrollTop / itemHeight));
};
```

Challenge 4: Variable row heights (the common follow-up)

Problem: With variable heights, `index × h` is wrong and `floor(scrollTop / h)` can't find the first row. Example: heights `[40, 120, 40, 80]`, `scrollTop = 170`. Row 1 spans 40..160 and is already above the viewport, so the first visible row is index 2 (160..200). `floor(170 / 70)` with an average height would say index 2 here only by luck.

Solution: Keep a **prefix sum of offsets** (`offsets[i]` = top of row i) and **binary search** it for `scrollTop`. Unmeasured rows use an estimated height; measure real rows with `ResizeObserver` and update the offsets.

```
// offsets = [0, 40, 160, 200, 280]  (length = count + 1, last = total height)
function findFirstIndex(offsets: number[], scrollTop: number) {
  let lo = 0, hi = offsets.length - 2;
  while (lo < hi) {
    const mid = (lo + hi + 1) >> 1;
    if (offsets[mid] <= scrollTop) lo = mid; else hi = mid - 1;
  }
  return lo; // findFirstIndex(offsets, 170) === 2
}
// row style: transform: translateY(offsets[index]px); spacer height: offsets[count]
```

O(log n) per scroll instead of O(n). This is what `react-virtuoso` / `@tanstack/react-virtual` do internally.

---

8. Implement in the correct order

#	Step	Working result after the step
1	`makeItems(100_000)`, render 50 of them in a plain `div`	Data shape confirmed on screen.
2	Fixed-height scroll container + spacer of full height	Scrollbar has the right size (still renders 50 rows).
3	`onScroll` → `scrollTop` state, compute `start/end`, render the slice with `translateY`	A working virtual list. Commit this first.
4	Extract `getVisibleRange` as a pure function; fix the partial-row `+1`; assert edge cases	Correct math at top, middle, bottom.
5	Add overscan	No blank strips on fast scroll.
6	Store `firstIndex` instead of `scrollTop`	Renders only when the row changes.
7	Extract `useVirtualList` and generic `VirtualList<T>` with `renderItem` / `getKey`	Reusable component.
8	`scrollToIndex` via `useImperativeHandle`; jump form with validation	Jump to row works, bad input shows an error.
9	Filter input with `useMemo`; reset to top on change; clamp stale index	Filter works with no blank frame.

#	Step	Working result after the step
10	Empty state	"No items match" message.
11	A11y: `tabIndex`, `role="list"`, `aria-setsize` / `aria-posinset`, `aria-live` count	Keyboard and screen reader friendly.
12	Styling and final walk through the use case table	Done.

9. Cover all states and edge cases

State / edge case	What happens
Initial	Top of the list, rows 0–14 rendered (10 visible + 5 overscan below).
Processing (filtering 100k)	Synchronous `filter` takes a few ms; fine for 100k. For heavier work: `useDeferredValue(query)`.
Success / scrolling	~20 rows in the DOM at any scroll position.
Empty list	`emptyMessage` rendered instead of the scroll container.
Error (bad jump input)	`role="alert"` message, list does not move.
Fewer items than fit the viewport (e.g. 3)	`end` clamps to 3; renders 3 rows, no scrollbar.
Scrolled to the very bottom	`firstIndex` = last page; last row fully visible; no rows past `itemCount`.
List shrinks while scrolled (filter)	Stale index clamped to the last page; parent also calls `scrollToIndex(0)`.
`scrollToIndex` out of range (-1, 1e9)	Clamped to `[0, count-1]`.
`itemHeight` is 0 or negative	Returns an empty range instead of dividing by zero.
Fast fling / scrollbar drag	Overscan hides most gaps; rows fill on the next frame.
Very large lists (1M × 56px = 56M px)	Over browser max element height (~17.9M px in Firefox, ~33.5M px in Chrome). Needs scaled scrolling (see Q&A).
Rows with focusable content scroll out	That row unmounts and focus is lost. Known limitation of virtualization (see Q&A).
Browser find (Ctrl+F)	Can't find off-screen rows; they are not in the DOM. Provide your own search (the filter).

10. Final structure

```
_28_virtulized_list/
├── index.html
├── package.json
├── vite.config.ts
```

```
└── src/
    ├── main.tsx
    ├── index.css
    ├── App.tsx                    // page: filter, jump form, count, list
    ├── App.css
    ├── data/
    │   └── makeItems.ts           // mock items with stable ids
    ├── utils/
    │   └── getVisibleRange.ts     // pure windowing math
    ├── hooks/
    │   └── useVirtualList.ts      // scroll state, range, scrollToIndex
    └── components/
        ├── VirtualList.tsx        // generic windowed list
        └── VirtualList.css
```

11. Full working code

`src/utils/getVisibleRange.ts`

```ts
export type Range = { start: number; end: number }; // end is exclusive

type RangeInput = {
  firstIndex: number; // first row touching the top edge of the viewport
  viewportHeight: number;
  itemHeight: number;
  itemCount: number;
  overscan: number;
};

export function getVisibleRange({
  firstIndex,
  viewportHeight,
  itemHeight,
  itemCount,
  overscan,
}: RangeInput): Range {
  if (itemCount <= 0 || itemHeight <= 0) return { start: 0, end: 0 };

  // A stale index (list just shrank) snaps to the last page instead of rendering nothing
  const maxFirst = Math.max(0, Math.floor(itemCount - viewportHeight / itemHeight));
  const first = Math.min(firstIndex, maxFirst);
  // +1: when scrolled mid-row, one extra row peeks in at the bottom
  const visibleCount = Math.ceil(viewportHeight / itemHeight) + 1;
  const end = Math.min(itemCount, first + visibleCount + overscan);
  const start = Math.max(0, first - overscan);
  return { start, end };
}
```

`src/hooks/useVirtualList.ts`

```ts
import { useRef, useState } from 'react';
import type { UIEvent } from 'react';
import { getVisibleRange } from '../utils/getVisibleRange';
```

```
type Options = {
  itemCount: number;
  itemHeight: number;
  height: number;
  overscan?: number;
};

export function useVirtualList({ itemCount, itemHeight, height, overscan = 5 }: Options) {
  const containerRef = useRef<HTMLDivElement>(null);
  // Store the first row index, not raw scrollTop: React skips the re-render
  // while the user scrolls within the same row.
  const [firstIndex, setFirstIndex] = useState(0);

  const onScroll = (e: UIEvent<HTMLDivElement>) => {
    setFirstIndex(Math.floor(e.currentTarget.scrollTop / itemHeight));
  };

  const scrollToIndex = (index: number) => {
    const safe = Math.max(0, Math.min(index, itemCount - 1));
    containerRef.current?.scrollTo({ top: safe * itemHeight });
    setFirstIndex(safe); // render the target rows now, don't wait for the scroll event
  };

  const range = getVisibleRange({
    firstIndex,
    viewportHeight: height,
    itemHeight,
    itemCount,
    overscan,
  });

  return { containerRef, onScroll, range, totalHeight: itemCount * itemHeight, scrollToIndex };
}
```

`src/components/VirtualList.tsx`

```
import { useImperativeHandle } from 'react';
import type { Key, ReactNode, Ref } from 'react';
import { useVirtualList } from '../hooks/useVirtualList';
import './VirtualList.css';

export type VirtualListHandle = { scrollToIndex: (index: number) => void };

type Props<T> = {
  items: T[];
  itemHeight: number;
  height: number;
  overscan?: number;
  getKey: (item: T) => Key;
  renderItem: (item: T, index: number) => ReactNode;
  emptyMessage?: ReactNode;
  ariaLabel: string;
  ref?: Ref<VirtualListHandle>;
};

export function VirtualList<T>({
  items,
```

```
  itemHeight,
  height,
  overscan,
  getKey,
  renderItem,
  emptyMessage = 'No items',
  ariaLabel,
  ref,
}: Props<T>) {
  const { containerRef, onScroll, range, totalHeight, scrollToIndex } = useVirtualList({
    itemCount: items.length,
    itemHeight,
    height,
    overscan,
  });

  useImperativeHandle(ref, () => ({ scrollToIndex }));

  if (items.length === 0) {
    return (
      <div className="vlist vlist--empty" style={{ height }}>
        {emptyMessage}
      </div>
    );
  }

  const visible = items.slice(range.start, range.end);

  return (
    <div
      ref={containerRef}
      className="vlist"
      style={{ height }}
      onScroll={onScroll}
      role="list"
      aria-label={ariaLabel}
      tabIndex={0}
    >
      {/* Spacer gives the scrollbar the full list height */}
      <div className="vlist__spacer" style={{ height: totalHeight }}>
        {visible.map((item, i) => {
          const index = range.start + i;
          return (
            <div
              key={getKey(item)}
              className="vlist__row"
              role="listitem"
              aria-setsize={items.length}
              aria-posinset={index + 1}
              style={{ height: itemHeight, transform: `translateY(${index * itemHeight}px)` }}
            >
              {renderItem(item, index)}
            </div>
          );
        })}
      </div>
    </div>
  );
}
```

`src/components/VirtualList.css`

```
.vlist {
  overflow-y: auto;
  position: relative;
  border: 1px solid #d0d5dd;
  border-radius: 8px;
  background: #fff;
}
.vlist:focus-visible {
  outline: 2px solid #4f46e5;
  outline-offset: 2px;
}
.vlist--empty {
  display: grid;
  place-items: center;
  color: #667085;
}
.vlist__spacer {
  position: relative;
}
.vlist__row {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  box-sizing: border-box;
}
```

`src/data/makeItems.ts`

```
export type ListItem = { id: string; title: string; subtitle: string };

const TAGS = ['Design', 'Frontend', 'Backend', 'Testing', 'DevOps'];

export function makeItems(count: number): ListItem[] {
  return Array.from({ length: count }, (_, i) => ({
    id: `item-${i}`,
    title: `Item ${i + 1}`,
    subtitle: `${TAGS[i % TAGS.length]} task`,
  }));
}
```

`src/App.tsx`

```
import { useMemo, useRef, useState } from 'react';
import type { FormEvent } from 'react';
import { VirtualList } from './components/VirtualList';
import type { VirtualListHandle } from './components/VirtualList';
import { makeItems } from './data/makeItems';
import type { ListItem } from './data/makeItems';
import './App.css';

const ALL_ITEMS = makeItems(100_000);
const ROW_HEIGHT = 56;
```

```
const LIST_HEIGHT = 480;

export default function App() {
  const listRef = useRef<VirtualListHandle>(null);
  const [query, setQuery] = useState('');
  const [jumpTo, setJumpTo] = useState('');
  const [jumpError, setJumpError] = useState('');

  const items = useMemo(() => {
    const q = query.trim().toLowerCase();
    return q ? ALL_ITEMS.filter((item) => item.title.toLowerCase().includes(q)) : ALL_ITEMS;
  }, [query]);

  const onQueryChange = (value: string) => {
    setQuery(value);
    listRef.current?.scrollToIndex(0); // new result set starts at the top
  };

  const onJump = (e: FormEvent) => {
    e.preventDefault();
    const n = Number(jumpTo);
    if (!Number.isInteger(n) || n < 1 || n > items.length) {
      setJumpError(`Enter a row between 1 and ${items.length.toLocaleString()}`);
      return;
    }
    setJumpError('');
    listRef.current?.scrollToIndex(n - 1);
  };

  return (
    <main className="app">
      <h1>Virtualized List</h1>

      <div className="controls">
        <input
          type="search"
          placeholder="Filter by title…"
          aria-label="Filter items"
          value={query}
          onChange={(e) => onQueryChange(e.target.value)}
        />
        <form onSubmit={onJump} className="jump">
          <input
            type="number"
            placeholder="Row #"
            aria-label="Jump to row"
            value={jumpTo}
            onChange={(e) => setJumpTo(e.target.value)}
          />
          <button type="submit">Go</button>
        </form>
      </div>
      {jumpError && (
        <p className="error" role="alert">
          {jumpError}
        </p>
      )}

      <p className="count" aria-live="polite">
        {items.length.toLocaleString()} items
```

```
      </p>

      <VirtualList<ListItem>
        ref={listRef}
        items={items}
        itemHeight={ROW_HEIGHT}
        height={LIST_HEIGHT}
        overscan={5}
        ariaLabel="Items"
        getKey={(item) => item.id}
        emptyMessage={`No items match "${query}"`}
        renderItem={(item) => (
          <div className="row">
            <strong>{item.title}</strong>
            <span>{item.subtitle}</span>
          </div>
        )}
      />
    </main>
  );
}
```

`src/App.css`

```
.app {
  max-width: 560px;
  margin: 0 auto;
  padding: 24px 16px;
}
.controls {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
}
.controls input {
  padding: 8px 10px;
  border: 1px solid #d0d5dd;
  border-radius: 6px;
  font: inherit;
}
.controls input[type='search'] {
  flex: 1;
  min-width: 180px;
}
.jump {
  display: flex;
  gap: 6px;
}
.jump input {
  width: 90px;
}
.jump button {
  padding: 8px 14px;
  border: 0;
  border-radius: 6px;
  background: #4f46e5;
  color: #fff;
  font: inherit;
```

```
  cursor: pointer;
}
.error {
  color: #b42318;
  margin: 6px 0 0;
}
.count {
  color: #667085;
  margin: 12px 0 8px;
}
.row {
  height: 100%;
  display: flex;
  flex-direction: column;
  justify-content: center;
  padding: 0 16px;
  border-bottom: 1px solid #eaecf0;
}
.row span {
  color: #667085;
  font-size: 0.85rem;
}
```

`src/index.css`

```
body {
  margin: 0;
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
  color: #101828;
  background: #f9fafb;
}
```

`src/main.tsx` (unchanged from the Vite template)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Self-check for the pure math (`node selfcheck.ts`, Node 22.6+ runs TypeScript directly)

```
import assert from 'node:assert/strict';
import { getVisibleRange } from './src/utils/getVisibleRange.ts';

const r = (firstIndex: number, itemCount = 1000, overscan = 0) =>
  getVisibleRange({ firstIndex, viewportHeight: 500, itemHeight: 50, itemCount, overscan });

assert.deepEqual(r(0), { start: 0, end: 11 });
assert.deepEqual(r(0, 1000, 5), { start: 0, end: 16 });
```

```
assert.deepEqual(r(100, 1000, 5), { start: 95, end: 116 });
assert.deepEqual(r(990, 1000, 5), { start: 985, end: 1000 }); // max scroll: 1000*50 - 500
assert.deepEqual(r(995, 1000, 5), { start: 985, end: 1000 }); // past the end snaps back
assert.deepEqual(r(0, 0), { start: 0, end: 0 });
assert.deepEqual(r(0, 3), { start: 0, end: 3 });
assert.deepEqual(r(5000, 10, 2), { start: 0, end: 10 }); // stale index after the list shrank
assert.deepEqual(r(5000, 19, 5), { start: 4, end: 19 }); // filter left 19 rows: last page, not blank
console.log('selfcheck ok');
```

---

12. Wrap-up

Edge cases handled

- Partial row at the bottom (`+1` row), top and bottom clamping, empty list, fewer rows than the viewport.
- Stale scroll index after the list shrinks: snaps to the last page instead of rendering nothing.
- `scrollToIndex` clamps any index and renders the target rows immediately.
- Invalid jump input (0, negative, decimal, empty, too large) shows an error.
- Zero / negative row height does not divide by zero.
- Keyboard scrolling and screen reader position info.

Trade-offs to mention

Decision	Why	Cost
Fixed row height	O(1) math, simplest correct solution in 60 min	Can't show wrapped text of different lengths
Store `firstIndex`, not `scrollTop`	One render per row instead of per pixel	Can't use sub-row scroll position in render (not needed with fixed heights)
Absolute rows + `translateY`	No layout shift, compositor-friendly	Rows need an explicit height
Native scrolling	Smooth, accessible, keyboard and scrollbar work for free	Bound by browser max element height
Imperative `scrollToIndex` handle	Parent controls scrolling without owning scroll state	Less "React-y" than a prop, but avoids feedback loops
Build it vs `react-window`	Shows understanding; no dependency	In production, a battle-tested library handles more edge cases

What to add with more time

- **Variable / dynamic heights:** prefix offsets + binary search + `ResizeObserver` measurement.
- **Auto height:** measure the container with `ResizeObserver` instead of a fixed `height` prop.
- **Infinite loading:** when `range.end` gets near `items.length`, fetch the next page; show skeleton rows.
- **Tests:** Vitest unit tests for `getVisibleRange`; React Testing Library for render count and `scrollToIndex`; a Playwright test that scrolls and checks DOM row count.

- **Performance:** `memo` rows, `useDeferredValue` for filter, `content-visibility`, profiling with React DevTools.
- **Keyboard row navigation:** roving focus with Arrow keys that scrolls the focused row into view (`role="listbox"` + `aria-activedescendant`).
- **Grid virtualization:** the same math on both axes.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why virtualize at all? What actually breaks with 100,000 plain rows?

Each row is several DOM nodes, so 100k rows is 300k+ nodes. The first render takes seconds, memory goes up by hundreds of MB, and every style recalc or layout touches all of them, so scrolling and typing lag. Virtualization keeps the DOM at ~20 rows, so cost is constant regardless of list size.

Q2. When would you NOT virtualize?

For small lists (a few hundred rows), when the page must be fully searchable with Ctrl+F or crawlable for SEO, or when printing. Virtualization adds complexity and breaks browser find, so only use it when the list is large enough to hurt. `content-visibility: auto` is a cheaper middle ground.

Q3. Why not just use `react-window` or `@tanstack/react-virtual`?

In production I would, they handle variable heights, RTL, horizontal lists and many edge cases. In the interview the point is to show I understand the mechanism (spacer, window math, overscan), and this solution uses the same API shape (`scrollToItem` via ref, `itemSize`, `overscanCount`).

State design

Q4. Why store `firstIndex` instead of `scrollTop`?

`scrollTop` changes every pixel, `firstIndex` changes once per 56px. React skips the re-render when `setState` gets the same value, so scrolling within a row costs nothing. With fixed heights the render only needs to know which row is first.

Q5. Why is `range` not in state?

It is fully determined by `firstIndex`, `height`, `itemHeight`, `itemCount` and `overscan`. Storing it means keeping two sources of truth in sync (for example when `items` changes length). Deriving it during render is cheap (a few arithmetic ops) and always correct.

Q6. Why does the parent call `scrollToIndex` through a ref instead of passing `scrollIndex` as a prop?

Scrolling is an event, not a state. With a prop, "jump to 777 twice" does nothing the second time (same value), and you need extra logic to sync the prop back when the user scrolls manually. An imperative handle says "do this now", which is what the user asked for.

Challenge 1: spacer and positioning

Q7. Why `transform: translateY` instead of `top`, or instead of padding spacers above and below?

`transform` doesn't trigger layout; the compositor moves the layer. Changing `top` or padding forces layout on every scroll render. Top/bottom padding spacers also work and keep rows in normal flow, but every range change resizes the padding, which is a layout change.

Q8. What if the data is 1 million items?

1M × 56px = 56M px, more than the browser max element height (about 33.5M px in Chrome, about 17.9M px in Firefox), so the spacer gets clamped and the bottom rows become unreachable. Fix: cap the spacer height (e.g. 10M px) and map scroll position with a ratio, `realOffset = scrollTop × (realTotal - viewport) / (cappedTotal - viewport)`. Also keep data in a typed array or fetch pages instead of holding 1M objects.

Challenge 2: math

Q9. Why `ceil(viewport / itemHeight) + 1`?

`ceil` covers a viewport that is not a multiple of the row height. The extra `+1` covers being scrolled mid-row: the top row is partly hidden and one more row peeks in at the bottom. Without it you see a white strip at the bottom while scrolling.

Q10. The list shrinks from 100,000 to 19 while I'm scrolled to row 50,000. What happens?

The browser clamps `scrollTop` and fires a scroll event, but that is async. For one render the index is stale. `getVisibleRange` clamps `firstIndex` to the last valid page, so it renders rows 4–18 instead of nothing, and the parent calls `scrollToIndex(0)` so the new results start at the top.

Challenge 3: performance

Q11. How big should overscan be?

It's a trade-off: more overscan means fewer blank flashes on fast scrolls, but more rows to render each time. 3–10 rows is typical. With expensive rows go lower; with cheap rows and fast flings go higher. Some libraries overscan more in the scroll direction only.

Q12. Should you throttle or debounce the scroll handler?

Debounce: no, the list would stay blank until scrolling stops. Throttling with `requestAnimationFrame` is fine but gains little here because scroll events already fire about once per frame, and storing `firstIndex` already removes the redundant renders. Scroll events can't be cancelled, so the handler never blocks the browser's scrolling; the only cost is the work done inside it, which here is one division and a `setState`.

Q13. Rows are expensive (images, charts). Scrolling is janky. What do you do?

Profile first. Then: `memo` the row component so rows that stay on screen don't re-render; keep `renderItem` and `getKey` stable; lazy-load images with fixed dimensions; render a lightweight placeholder while scrolling fast (`isScrolling` flag) and the full row once it settles; increase overscan slightly.

Q14. What if the filter is slow on 1M items?

Wrap the query with `useDeferredValue` so typing stays responsive and the filter runs at lower priority, or debounce it. For really large data move filtering to a Web Worker or the server. Pre-lowercase the searchable text once instead of on every keystroke.

Challenge 4: variable heights

Q15. How do you handle rows whose height you only know after render?

Start with an estimated height per row, build prefix offsets, and binary-search them for the first visible row. After a row mounts, measure it with `ResizeObserver`, update its height and recompute offsets from that index on. If a row above the viewport changes height, adjust `scrollTop` by the difference so the content doesn't jump (scroll anchoring).

Q16. Why binary search? Why not loop through offsets?

A linear scan to row 500,000 is 500k steps on every scroll event. The offsets array is sorted, so binary search finds the first row in about 20 steps. Updating offsets after a measurement is O(n) from that index; a Fenwick tree makes both operations O(log n) if needed.

Implementation steps

Q17. What happens to focus and input state when a row scrolls out?

The row unmounts, so focus is lost and any uncontrolled input state is gone. Keep row state in the parent (keyed by item id), not inside the row. For keyboard navigation, use `aria-activedescendant` on the container or make sure the focused row is always rendered (keep it in the rendered set even outside the range).

Q18. Why key by `item.id` and not by index?

With index keys, after filtering, the row at slot 0 shows a different item but React reuses the old DOM and any state inside it, so you can see stale content. With id keys, each item keeps its own row. Keying by position in the visible slice is worse: every scroll shifts which item sits at slot 0, so every row's props change and every row re-renders.

Q19. How would you make it accessible?

The container is focusable (`tabIndex={0}`) so arrow keys, PageDown and End scroll it natively. `role="list"` with `role="listitem"` rows, plus `aria-setsize` and `aria-posinset` so a screen reader says "item 772 of 100,000" even though only 20 rows exist. The item count uses `aria-live="polite"`; the jump error uses `role="alert"`.

What-if twists

Q20. What if the data comes from a slow paginated API?

Get `total` first so the spacer has the full height. Keep a page cache keyed by page number. When `range` covers pages not loaded, fetch them (dedupe in-flight requests, abort ones scrolled past with `AbortController`) and render skeleton rows meanwhile. Infinite scroll is the simpler version: fetch the next page when `range.end` is within a few rows of `items.length`.

Q21. What if the container height is not fixed (fills the window)?

Measure it: a `ResizeObserver` on the container stores `clientHeight` in state, and the hook uses that instead of the `height` prop. Resizing the window then just recomputes the range. Until the first measurement, render a reasonable default or nothing.

Q22. How would you test this?

Unit tests on `getVisibleRange` (top, middle, bottom, empty, short list, stale index), as in the self-check. Component tests: render 100k items and assert only ~15 `listitem`s exist; set `scrollTop`, fire `scroll`, assert the first row's text; call `scrollToIndex` and check rows. jsdom has no layout, so the math must not depend on measuring; an end-to-end Playwright test covers real scrolling.

Q23. How would you extend this to a grid (rows and columns)?

Run the same math on both axes: `getVisibleRange` once with `scrollTop` / row height and once with `scrollLeft` / column width, then render the cross product of visible rows and columns, each positioned with `translate(x, y)`. The spacer gets both full width and full height.